

EC-341 Digital Systems Design

Experiment # 13

Implementation of Clock Divider Circuit

1. Objectives:

- Clock Divider Circuit
- Lab Tasks

2. Theory

In previous labs, we have designed simple circuits of State Machines using Verilog Behavioral Modeling. Those circuits were driven by a single clock signal of 50MHz frequency. There are several cases in FPGA based systems where the operating frequencies of some components differ from others. Most of the time there is one core clock signal is available and the designer has to use this signal to create clock signals of other frequencies. The goal of this task is to use main clock signal to create signal of half of its frequency.

The technique is basic enough; we count the pulses from the main clock and trigger the second counter when a threshold is met.

Sequential circuits need a reliable clock signal to provide a stable time base for circuit operations. Clock signals originate from oscillator circuits that use a quartz crystal or silicon resonator to provide a stable frequency source. Driver circuits amplify the crystal (or oscillator) waveform into a digital signal that is distributed to the memory devices in a digital system.

Most digital systems need several (or even dozens of) different clock signals to drive different subsystems. For example, an FPGA-based system might use a 48 KHz clock to produce an audio stream, a 1 KHz clock to run a timer, a 10 MHz clock to run a small processor, and a 12 KHz clock to run a motor controller. It would be too expensive to use separate external oscillator circuits to create so many different clock signals, so systems typically produce the clocks they need from just one or two main clock inputs. The Blackboard, for example, uses just a single 100 MHz external clock source.

A clock divider circuit creates lower frequency clock signals from an input clock source. The divider circuit counts input clock cycles, and drives the output clock low and then high for some number of input clock cycles. For example, a clock divider

could drive an output low while counting five 100 MHz input clock cycles, then drive the signal high while counting another five cycles, and so on in an endlessly repeating sequence to create a 10 MHz clock.

Note that in an FPGA (like the ZYNQ chip), clock signals that drive flip-flops can only come from two sources: the main clock input, or directly from the output of a flip-flop. A clock signal cannot come from any sort of logic gate or logic circuit.

A simple “divide by 2” clock divider can use a single flip-flop as shown below. Note that in the Verilog code, the output “clk_div” is typecast to a reg in the module port statement. This must be done because “clk_div” is assigned in a procedural assignment statement, and all procedural assignment targets must be reg type. It is also worth mentioning that module outputs can be type wire or reg, but inputs must be type wire.

50MHz clock frequency to 50Hz

```

module clk_div(
    input rst,
    input clk,
    output reg s_clk,
    output reg [20:0] count
);
always @ (posedge clk or posedge rst)
begin
    if(rst)
        begin
            count<=0;
            s_clk<=0;
        end

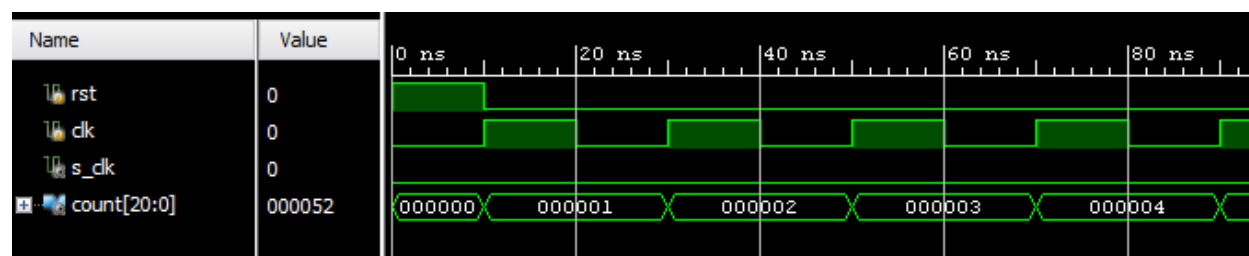
    else if(count<=500000)
        count=count+1;
    else
        begin
            s_clk<=~s_clk;
            count<=0;
        end
    end
end
endmodule

```

```

module test_clk_div(
);
    reg rst;
    reg clk;
    wire s_clk;
    wire [20:0] count;
    clk_div TB_clk_div(rst,clk,s_clk,count);
    always
        #10 clk=~clk;
    initial begin
        rst=1;
        clk=0;
        #10 rst=0;
    end
endmodule

```





3. Tasks

1. Write a code which takes as an input one main clock and generates four output clock signals each half the frequency of the previous one.
2. Write a top level module, which integrates the above module and a smaller counter module instantiated four times operating at four different clock frequencies generated by the above module.

